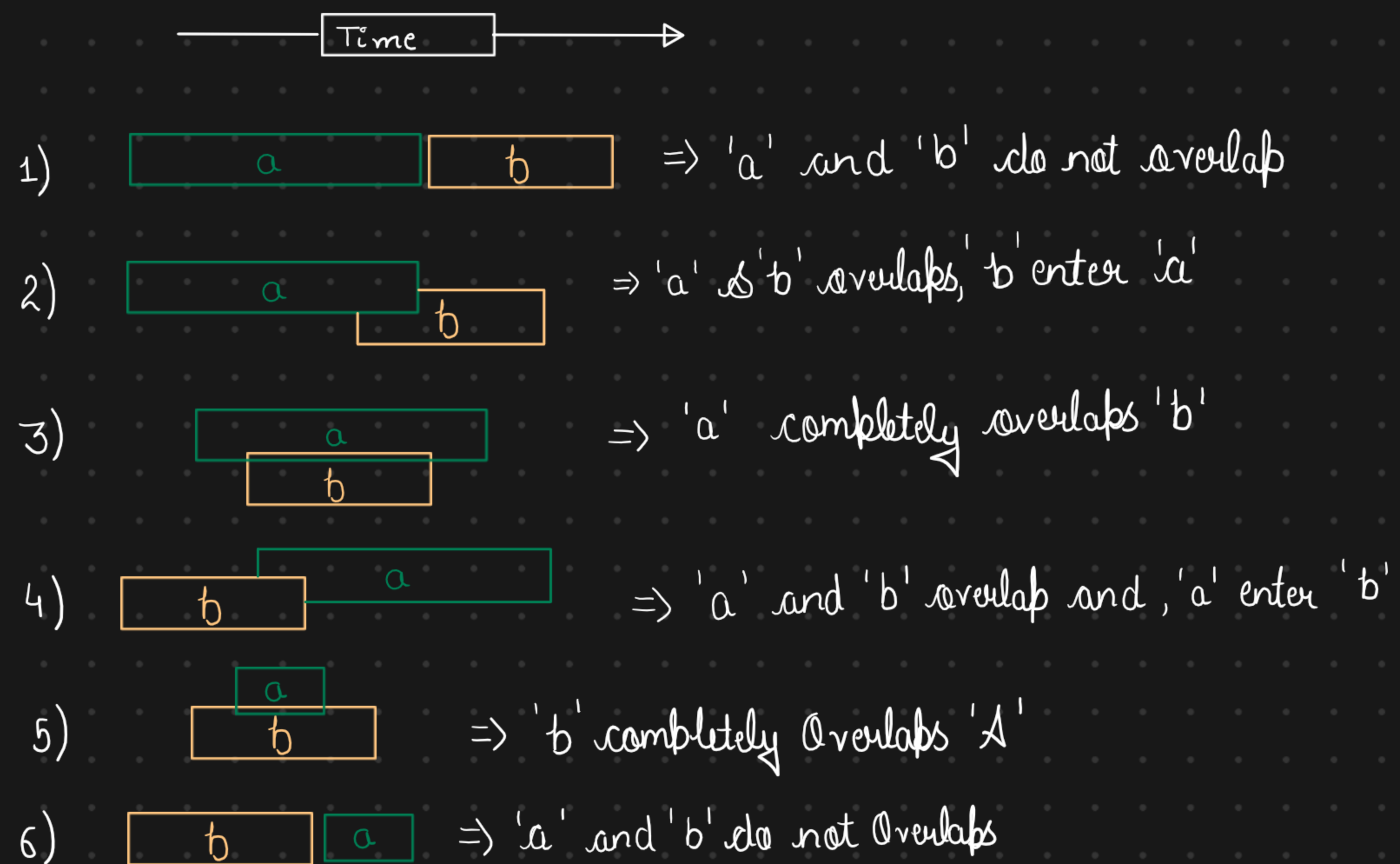


Merge Intervals

This patterns describes an efficient technique to deal with overlapping intervals. In a lot of problem involving intervals, we either need overlapping interval or merge interval if they overlap.

given two interval ('a' and 'b') there will be six different ways the two interval can relate to each other.



Problem - Merge Intervals

Problem Statement

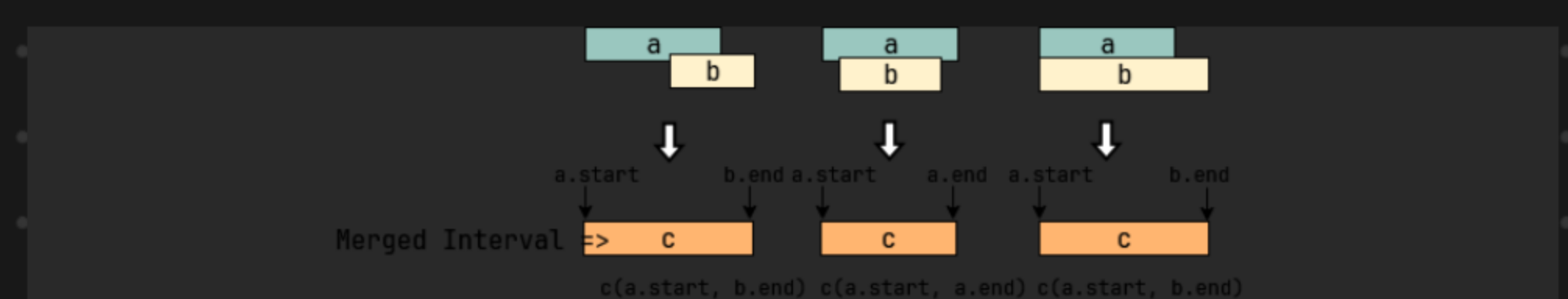
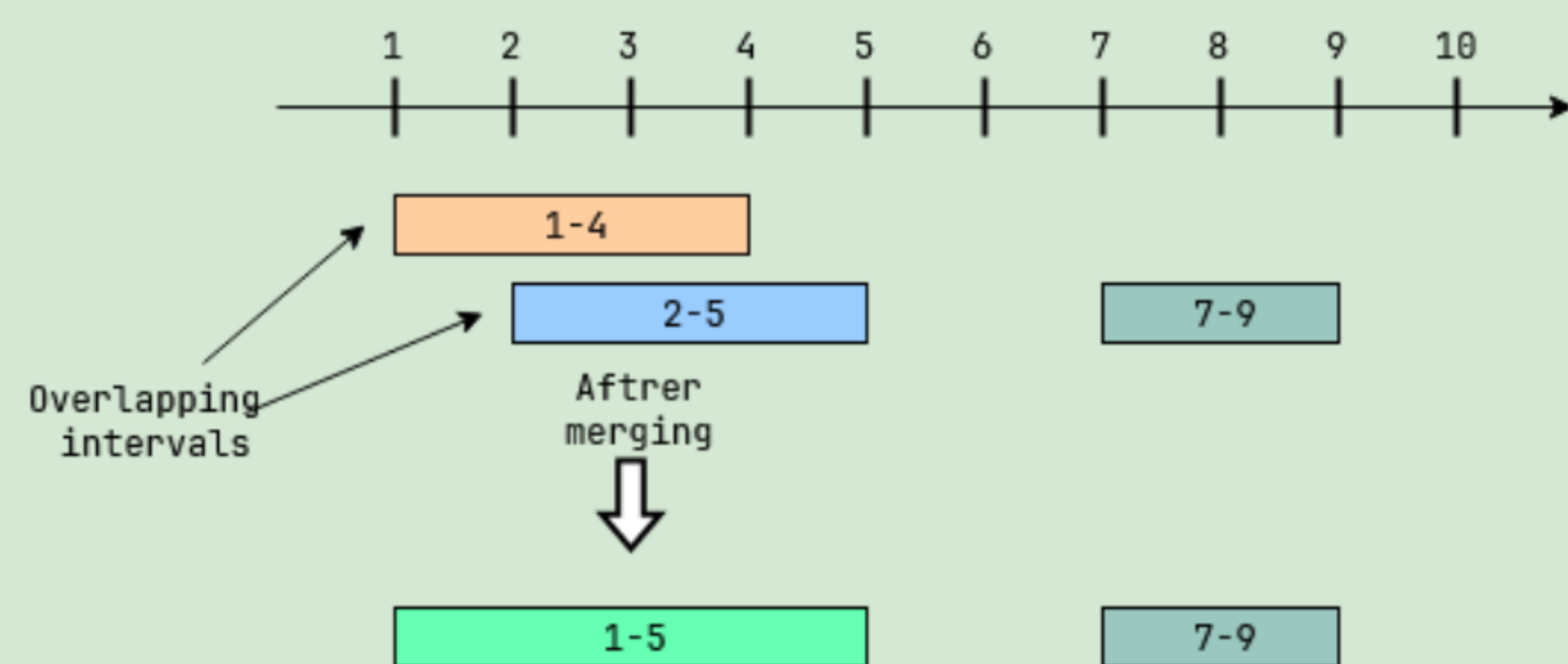
Given a list of intervals, merge all the overlapping intervals to produce a list that has only mutually exclusive intervals.

Example 1:

Intervals: [[1,4], [2,5], [7,9]]

Output: [[1,5], [7,9]]

Explanation: Since the first two intervals [1,4] and [2,5] overlap, we merged them into one [1,5].



```
#include <iostream>
using namespace std;
```



```
class Interval {
```

```
public:
```

```
int start = 0;
```

```
int end = 0;
```

```
Interval(int start, int end) {
```

```
    this->start = start;
```

```
    this->end = end;
```

```
}
```

```
};
```

```
class MergeIntervals {
```

```
public:
```

```
static vector<Interval> merge(vector<Interval>
```

```
    intervals) {
```

```
    if (intervals.size() < 2) {
```

```
        return intervals;
```

```
    }  
    // Sort the intervals by start time.
```

```
    sort(intervals.begin(), intervals.end(), [](const Interval &x,  
        const Interval &y) { return x.start < y.start; });
```

```
    vector<Interval> mergedIntervals;
```

```
    vector<Interval>::const_iterator intervalItr = intervals.begin();
```

```
    Interval interval = *intervalItr++;
```

```
    int start = interval.start;
```

```
    int end = interval.end;
```

```
    while (intervalItr != intervals.end()) {
```

```
        interval = *intervalItr++;
```

```
        if (interval.start <= end) {
```

```
            end = max(interval.end, end);
```

```
        } else {
```



```

        mergedIntervals.push_back({start, end});
        start = interval.start;
        end = interval.end;
    }
}
mergedIntervals.push_back({start, end});
return mergedIntervals;
}
};

```

A cleaner version

```

class MergeIntervals {

```

```

public:

```

```

    static vector<Interval> merge(vector<Interval> & intervals) {

```

```

        if (intervals.size() < 2) return intervals;

```

```

        // Sort by start time

```

```

        sort(intervals.begin(), intervals.end(), [](const Interval & a, const
            Interval & b) {
            return a.start < b.start;
        });

```

```

        vector<Interval> merged;

```

```

        Interval current = intervals[0];

```

```

        for (const interval & : intervals) {

```

```

            if (interval.start <= current.end) {

```

```

                // Overlapping so merge

```

```

                current.end = max(current.end, interval.end);

```

```

            } else {

```

```

                // No overlap, push current and reset

```

```

                merged.push_back(current);

```



```

        current = interval;
    }
}
// Push the last interval
merge.push_back(current);
return merged;
}
};

```

Problem - Insert Interval (medium)

Problem Statement

Given a list of non-overlapping intervals sorted by their start time, insert a given interval at the correct position and merge all necessary intervals to produce a list that has only mutually exclusive intervals.

Example 1:

Input: Intervals=[[1,3], [5,7], [8,12]], New Interval=[4,6]
 Output: [[1,3], [4,7], [8,12]]
 Explanation: After insertion, since [4,6] overlaps with [5,7], we merged them into one [4,7].

Example 2:

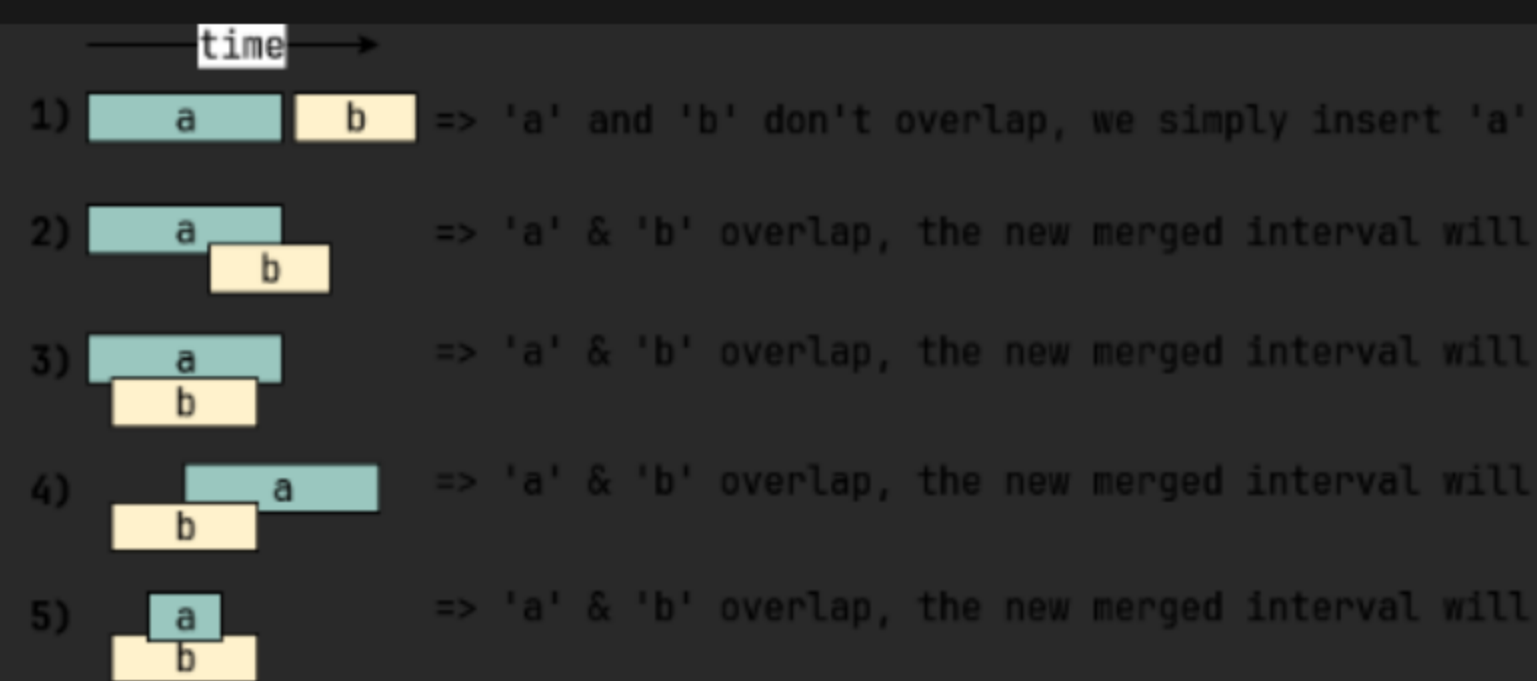
Input: Intervals=[[1,3], [5,7], [8,12]], New Interval=[4,10]
 Output: [[1,3], [4,12]]
 Explanation: After insertion, since [4,10] overlaps with [5,7] & [8,12], we merged them into one [4,12].

Solution → if the given list was not sorted, we could have simply appended the new interval to it and used the `merge()` function from Merge Interval. Since the given list is sorted, we should try to come up with a solution better than $O(N * \log N)$.

When inserting a new interval in a sorted list, we need to first find the correct index where the new interval can be placed. In other words, we need to skip all the intervals which end before the start of the new interval. So we can iterate through the given sorted list of intervals and skip all the intervals with the following condition:

`intervals[i].end < newInterval.start`

Once we have found the correct place, we can follow an approach similar to Merge Interval to insert and/or merge the new interval 'a' and the first interval with the above condition 'b'. There are five possibilities:



The diagram above clearly shows the merging approach, To handle all four merging scenarios

$c.start = \min(a.start, b.start)$

$c.end = \max(a.end, b.end)$

```
#include <iostream/ stdc++.h>
```

```
using namespace std;
```

```
class Interval {
```

```
public:
```

```
int start = 0;
```

```
int end = 0;
```

```
Interval(int start, int end) {
```

```
    this->start = start;
```

```
    this->end = end;
```

```
}
```

```
};
```

```
class InsertInterval {
```

```
public:
```

```
static vector<Interval> insert(const vector<Interval>& intervals,  
                                Interval newInterval) {
```

```
    if (intervals.empty()) {
```

```
        return vector<Interval> {newInterval};
```

```
}
```

```
vector<Interval> mergedIntervals;
```

```
int i = 0;
```

// skip (and add to output) all intervals that come before the 'newInterval'.


```

        while ( i < intervals.size() && intervals[i].end < newInterval.start){
            mergedIntervals.push_back(intervals[i++]);
        }
        //merge all intervals that overlap with 'newInterval'
        while ( i < intervals.size() && intervals[i].start <= newInterval.end){
            newInterval.start = min(intervals[i].start, newInterval.start);
            newInterval.end = max(intervals[i].end, newInterval.end);
            i++;
        }
        // insert the newInterval
        mergedIntervals.push_back(newInterval);
        // add all the remaining intervals to the output
        while ( i < intervals.size()){
            mergedIntervals.push_back(intervals[i++]);
        }
        return mergedIntervals;
    }
};

```

Problem - Intervals Intersection

Given two lists of intervals, find the intersection of these two lists. Each list consists of disjoint intervals sorted on their start time.

Example 1:

Input: arr1=[[1, 3], [5, 6], [7, 9]], arr2=[[2, 3], [5, 7]]
 Output: [2, 3], [5, 6], [7, 7]
 Explanation: The output list contains the common intervals between the two lists.

Example 2:

Input: arr1=[[1, 3], [5, 7], [9, 12]], arr2=[[5, 10]]
 Output: [5, 7], [9, 10]
 Explanation: The output list contains the common intervals between the two lists.

Try it yourself #

Try solving this question here:

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Interval {

```

```

    public:

```

```

        int start = 0;

```

```

        int end = 0;

```



```

Interval(int start, int end){
    this->start = start;
    this->end = end;
}
};

class IntervalsIntersection {
public:
    static vector<Interval> merge(const vector<Interval> &arr1, const vector<Interval> &arr2){
        vector<Interval> result;
        int i = 0;
        int j = 0;
        while(i < arr1.size() && j < arr2.size()){
            // check if the interval arr[i] intersects with arr2[j]
            // check if one of the interval's start time lies within the other
            // interval
            if((arr1[i].start >= arr2[j].start && arr1[i].start <= arr2[j].end) || or
                (arr2[j].start >= arr1[i].start && arr2[j].start <= arr1[i].end)){
                // store the intersection part
                result.push_back({max(arr1[i].start, arr2[j].start), min(arr1[i].end,
                    arr2[j].end)});
            }
            // move next from the interval which is finishing first
            if(arr1[i].end < arr2[j].end){
                i++;
            } else {
                j++;
            }
        }
        return result;
    }
};

```


Problem - Conflicting Appointments

Problem Statement

Given an array of intervals representing 'N' appointments, find out if a person can attend all the appointments.

Example 1:

Appointments: [[1,4], [2,5], [7,9]]
Output: false
Explanation: Since [1,4] and [2,5] overlap, a person cannot attend both of these appointments.

Example 2:

Appointments: [[6,7], [2,4], [8,12]]
Output: true
Explanation: None of the appointments overlap, therefore a person can attend all of them.

Example 3:

Appointments: [[4,5], [2,3], [3,6]]
Output: false
Explanation: Since [4,5] and [3,6] overlap, a person cannot attend both of these appointments.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Interval{
```

```
public:
```

```
    int start = 0;
```

```
    int end = 0;
```

```
    Interval(int start, int end){
```

```
        this->start = start;
```

```
        this->end = end;
```

```
    }
```

```
};
```

```
class ConflictingAppointments {
```

```
public:
```

```
    static bool canAttendAllAppointments(vector<Interval> & intervals){
```

```
        // sort the intervals by start time.
```

```
        sort(intervals.begin(), intervals.end(), [](const Interval & x,  
                                                    const Interval & y){ return x.start < y.start; });
```

```
        // find any overlapping appointment
```



```

for( int i=1; i< intervals.size(); i++) {
    if( intervals[i].start < intervals[i-1].end){
        // please note the comparison above, it is '<' & not '<='
        // while merging we needed "<=" comparison, as we will
        // be merging the two intervals having condition
        // intervals[i].start == intervals[i-1].end" but such
        // intervals don't represent conflicting appointments as one
        // starts right after the other.
        return false;
    }
}
return true;
};

```

Problem Statement - 1

Given a list of intervals representing the start and end time of 'N' meetings, find the minimum number of rooms required to hold all the meetings.

Example 1:

Meetings: [[1,4], [2,5], [7,9]]

Output: 2

Explanation: Since [1,4] and [2,5] overlap, we need two rooms to hold these two meetings. [7,9] can occur in any of the two rooms later.

Example 2:

Meetings: [[6,7], [2,4], [8,12]]

Output: 1

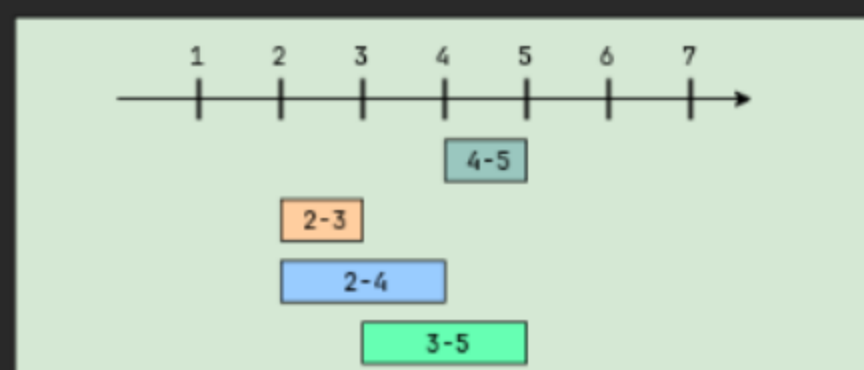
Explanation: None of the meetings overlap, therefore we only need one room to hold all meetings.

Meetings: [[4,5], [2,3], [2,4], [3,5]]

Output: 2

Explanation: We will need one room for [2,3] and [3,5], and another room for [2,4] and [4,5].

Here is a visual representation of Example 4:



Solution -

Let's take the above-mentioned example

Meeting $\rightarrow$ [[4,5], [2,3], [2,4], [3,5]]

Step - 1) Sorting these meeting on their start time will give us:
[[2,3],[2,4],[3,5],[4,5]]

Step - 2) Merging overlapping meeting

- [2,3] overlap with [2,4], so after merging we'll have $\Rightarrow$ [[2,4],[3,5],[4,5]]

- [2,4] overlap with [3,5], so after merging we'll have $\Rightarrow$ [[2,5],[4,5]]

- [2,5] overlaps [4,5], so after merging we'll have $\Rightarrow$ [2,5]

Since all the given meeting have merged into one big meeting ([2,5]), does this mean that they all overlapping and we need a minimum of four room to hold these meeting? you might have already guessed that the answer is No! as we can clearly see, some meeting are mutually exclusive. For example, [2,3] and [3,5] do not overlap and can happen in one room. So, to correctly solve our problem, we need to keep track of the mutual exclusiveness of the overlapping meeting.

here is what our strategy will look like:

- 1) We will sort the meeting based on start time.

- 2) We will schedule the first meeting (let call it M_1) in one room (lets call it r_1).

- 3) if the next meeting m_2 is not overlapping with m_1 , we can safely schedule it in the same room r_1 .

- 4) if the next meeting m_3 is overlapping with m_2 can't use r_1 , so we will schedule it in another room (let's call it r_2).

- 5). Now if the next meeting m_4 is overlapping with m_3 , we need to see if the room r_1 has become free. for this, we need to keep track of the meeting happening in it. if the end time of m_2 is before the start time of m_4 , we can use that room r_1 , otherwise we need to schedule m_4 in another room r_3 .

We can conclude that we need to keep track of all the meeting currently happening so that when we try to schedule a new meeting have already ended. We need to put this information in a data structure that can easily give us the smallest ending time. A Min heap would fit our requirement best.

So our algorithm will look like this:

1. Sort all the meeting on their start time.
2. Create a min-heap to store all the active meeting, This min-heap will also be used to find the active meeting with the smallest end time.
- 3) Iterate through all the meeting one by one to add them in the min-heap. let say we are trying to schedule the meeting m_1 .
- 4). Since the min-heap contains all the active meeting, so before scheduling m_1 we can remove all meeting from the heap that have an end time smaller than or equal to the start time of m_1 .
- 5) Now add m_1 to the heap.
- 6) The heap will always have all the overlapping meetings, so we will need rooms for all of them. keep a counter to remember size of the heap at any time which will be the minimum number of room needed.


```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Meeting {
```

```
public:
```

```
    int start = 0;
```

```
    int end = 0;
```

```
    Meeting(int start, int end) {
```

```
        public:
```

```
            int start = 0;
```

```
            int end = 0;
```

```
            Meeting(int start, int end) {
```

```
                this->start = start;
```

```
                this->end = end;
```

```
            }
```

```
    };
```

```
class MinimumMeetingRooms {
```

```
public:
```

```
    struct endCompare {
```

```
        bool operator()(const Meeting &x, const Meeting &y)
```

```
        { return x.end > y.end; }
```

```
    };
```

```
    static int findMinimumMeetingRoom(vector<Meeting> &meetings) {
```

```
        if (meetings.empty()) {
```

```
            return 0;
```

```
        }
```

```
        // sort the meetings by start time
```

```
        sort(meetings.begin(), meetings.end(), [](const Meeting &x,
```



```
const Meeting &y) {return x.start < y.start;});
```

```
int minRooms = 0;
```

```
priority_queue<Meeting, vector<Meeting>, endCompare> minHeap;
```

```
for (auto meeting : meetings) {
```

```
    // remove all meetings that have ended.
```

```
    while (!minHeap.empty() && meeting.start > minHeap.top().end) {
```

```
        minHeap.pop();
```

```
    }
```

```
    // add the current meeting into the minHeap
```

```
    minHeap.push(meeting);
```

```
    // all active meeting are in the minHeap, so we need  
    room for all of them.
```

```
    minRooms = max(minRooms, (int) minHeap.size());
```

```
}
```

```
return minRooms;
```

```
}
```

```
};
```